



Chapter 18

Assembly of Ancient Mitochondrial Genomes Without a Closely Related Reference Sequence

Christoph Hahn

Abstract

Recent methodological advances have transformed the field of ancient DNA (aDNA). Basic bioinformatics skills are becoming essential requirements to process and analyze the sheer amounts of data generated by current aDNA studies and in biomedical research in general. This chapter is intended as a practical guide to the assembly of ancient mitochondrial genomes, directly from genomic DNA-derived next-generation sequencing (NGS) data, specifically in the absence of closely related reference genomes. In a hands-on tutorial suitable for readers with little to no prior bioinformatics experience, we reconstruct the mitochondrial genome of a woolly mammoth deposited ~45,000 years ago. We introduce key software tools and outline general strategies for mitogenome assembly, including the critical quality assessment of assembly results without a reference genome.

Key words Ancient DNA, Genome assembly, Bioinformatics, Mitogenomics, Next-generation sequencing

1 Introduction

The emergence of next-generation sequencing (NGS) technologies has revolutionized evolutionary biology. NGS approaches as implemented by Illumina, Pacific Biosciences (PB), and Oxford Nanopore (ONP) sequencing platforms generate massive amounts of sequence data at low cost, democratizing genomics research. The field of ancient DNA has also been transformed by the recent methodological advances, with large-scale genomic data yielding new insights into the evolutionary histories of extinct species and enigmatic ancestors of the extant organismal diversity [1–5]. Ancient mitogenomics in particular has become a pillar of modern ancient DNA research, thanks to the higher representation of mitochondrial DNA compared to nuclear DNA in genomic data sets, the small size of most mitochondrial genomes, and the comparatively simple structure and inheritance pathway of mitochondrial genomes [6–12].

The most widely used NGS platform in the aDNA field is Illumina, which is characterized by generating short (<300 bp) sequence reads of high quality. Processing and analyzing the large amounts of data produced by the Illumina platforms requires specialized software, hardware, and skills. Many ancient DNA research projects begin by assembling a reference *de novo* mitochondrial genome. *De novo* genome assembly is the process of arranging short sequence reads into long contiguous nucleotide sequences based exclusively on overlap information present in sequence reads. A reliable reference genome can facilitate, for example, hybridization probe design for enrichment [13, 14], which is often necessary in downstream ancient DNA research given the generally poor preservation of ancient remains and common co-extraction of significant amounts of DNA from microbial and other contaminant organisms.

The maximum resolution a *de novo* genome assembly can reach is in principle limited by the maximum fragment and read lengths that can be obtained from the DNA extract compared to the length of repetitive elements in the target genome [15]. Thus, the relatively small size of DNA fragments that can be recovered from ancient DNA make *de novo* assembly particularly challenging for ancient DNA. Nonetheless, the comparably small size (typically 15–19 kbp) and modest complexity of mitochondrial genomes compared to nuclear genomes make mitochondrial genomes relatively accessible for *de novo* genome assembly.

Mitochondrial genomes are also more accessible than nuclear genomes for *de novo* assembly because of their high copy number in a DNA extract compared to nuclear genomes. This facilitates the so-called “genome-skimming” strategies for generating data for mitochondrial genome assembly, which rely on the generation of relatively small amounts of read data from genomic DNA extracts. Given that fragments in a DNA extract are sequenced at random, each mitochondrial nucleotide in a genomic DNA extract may be recovered by orders of magnitude more sequence reads than any nuclear base, with the exact ratio of mitochondrial to nuclear coverage depending on genome size and copy number ratios between nuclear and mitochondrial DNA in the sample.

Here, I outline strategies for the assembly of mitochondrial genomes in the absence of a closely related reference genome, directly from genomic DNA derived from Illumina sequencing, and the critical quality assessment of assembly results obtained from such data. In the tutorial below, I use the MITObim pipeline [16] and MIRA assembler [17] to assemble the mitochondrial genome of a woolly mammoth deposited at Oymyakon, District of Yakutia, in northeastern Siberia ~45,000 years ago [4]. The genomic sequence data were obtained using the Illumina HiSeq 2500 sequencing platform and made available upon publication of

the original manuscript [4]. The workflow outlined below is applicable to any genomic DNA sequence data.

2 Materials

2.1 Sequence Data

Illumina read data come in the FASTQ format, which combines nucleotide sequences and the associated per base quality scores [18]. Individual read lengths are typically in the range of 150–300 bp, sequenced from the two ends of fragments of length ~300–700 bp, either as single-end or paired reads. Preprocessing of Illumina data comprises a number of steps, including quality control and filtering, error correction, and potentially merging of paired reads. The read data used in the tutorial below are available to the public via the European Nucleotide Archive (accession number: ERR852928). Instructions on how to download the data can be found below.

2.2 Hardware

The below tutorial will run on any desktop computer or notebook with a minimum of 8 GB of memory. A network connection will be required for downloading the test data and necessary software. Completing the tutorial will require a total of 35 GB of disk space. The tutorial has been developed on a Dell OptiPlex 7040, Intel® Core™ i7-6700 CPU at 3.40 GHz, 8 threads, 32 GB RAM. Generally, NGS platforms generate large amounts of data (typically tens to hundreds of GB of raw data), which may, depending on the desired analyses, require specialized hardware. Basic analyses will be possible on reasonably modern desktop computers (4–8 cores, 8–16 GB of memory, ~500 GB disk space). Redundancy created during preprocessing of raw data can be expected to at least double to triple the raw data volume (at least temporarily). De novo genome assembly of large complex genomes in particular may require large amounts of memory (>500 GB) and specialized computing infrastructure.

2.3 Software

Generally, most bioinformatics software runs on UNIX-based systems such as Linux or Mac OS. The below tutorial was developed and tested on Ubuntu 16.04 (64-bit), and all software used is freely available. A detailed list of software tools and versions (Table 1) used in the tutorial as well as instructions for installation can be found below. All required software (except the graphical tool Tablet) has also been bundled in a dedicated Docker container (https://hub.docker.com/r/chrishah/amito_mimb/). Docker (<https://www.docker.com>) provides installation packages that facilitate integration with Windows and Mac operating systems and should thus enable operating system independent usage of the workflow demonstrated.

The software used in this tutorial is described in Table 1.

Table 1
Detailed list of software used in the tutorial (all last accessed 16 Jan 2019)

Software package	Version	Available from URL
BLAST+	2.2.31+	ftp://ftp.ncbi.nlm.nih.gov/blast/executables/LATEST/
MITObim	1.9.1	https://github.com/chrishah/MITObim
MIRA	4.0.2	https://sourceforge.net/projects/mira-assembler/
MUSCLE	3.8.31	http://drive5.com/muscle/
Perl	5.22.1	http://www.perl.org/
Python	2.7.12	https://www.python.org/
Tablet	1.16.09.06	https://ics.hutton.ac.uk/tablet/

3 Methods

The following tutorial details strategies for assembling the mitochondrial genome of a woolly mammoth (*Mammuthus primigenius*) directly from genomic sequence read data using the MITObim pipeline [16] (see **Note 1**). The MITObim pipeline is designed to reconstruct complete mitochondrial genomes from genomic short-read sequence data without the need for a close reference genome. The pipeline initially identifies genuine mitochondrial reads based on sequence similarity to mitochondrial bait sequences (full mitochondrial genomes or short fragments) from potentially distantly related species [16]. These reads are then assembled using the MIRA assembler [17]. The process is repeated iteratively using the assembly result from the previous iteration as bait sequence to identify more mitochondrial reads. Given the circular nature of the mitochondrial genome, the number of reads baited is expected to plateau once the molecule is fully reconstructed. In the tutorial, I use mitochondrial sequence data from the West Indian manatee (*Trichechus manatus*) as bait. The most recent common ancestor (MRCA) of *M. primigenius* and *T. manatus* is thought to have lived in the late Cretaceous, some 75 million years ago [19]. Theoretically, one single short region of length k basepairs that is identical between the bait sequence and the mitochondrial genome in question is sufficient to seed a robust and successful mitochondrial reconstruction (Fig. 1).

The tutorial is mainly command line based. Basic prior experience with the Unix shell is advantageous but not necessary for successful completion of the tutorial. Below, I provide code cells detailing commands to be executed in the Unix shell. “\$” symbols at the beginning of lines symbolizes the prompt of the Unix shell and are not part of the command. “\” symbols are often inserted at

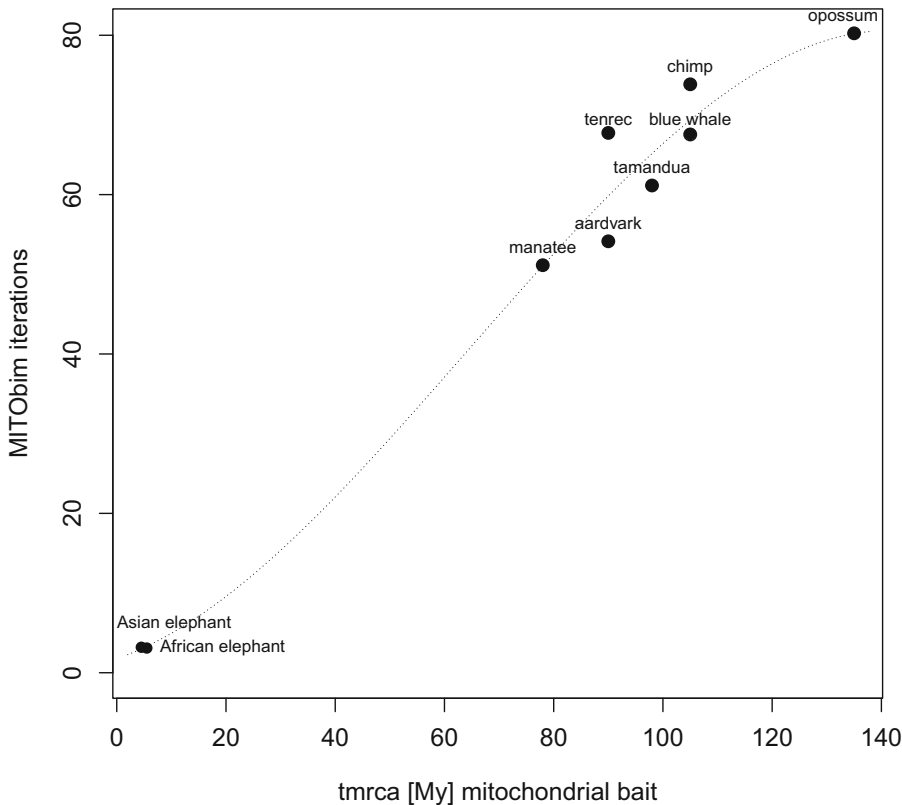


Fig. 1 Number of MITObim iterations required for reconstructing the woolly mammoth mitochondrial genome (average across ten randomly drawn subsets of 20% of the read data, ENA accession: ERR852928) dependent on the divergence from the mitochondrial reference. *tmrca* time most recent common ancestor (estimates from ref. 19)

the end of command lines to increase readability and indicate the continuation of the same command in the next line. You may write out lines linked via “\” in single one-line command. Lines starting with “#” indicate comments that are intended to help the user understand the following code.

The tutorial makes extensive use of variables; in the Unix shell, these are first defined using the syntax “variable=sometext” or “variable=10” and subsequently called via “\$variable”. While this may at first appear complicated to users with little prior experience in bioinformatics, the intention is to make the strategy described in this chapter easily applicable to any other data set. To apply the same workflow to other data, simply adjust the variable declarations at the start of the session (e.g., filenames and file locations, etc.), and, in principle, no further changes should be needed to the actual commands. However, successful reconstruction of mitochondrial genomes from short-read data will depend on the data (depth of coverage, quality) and the characteristics of the particular genome, such as GC content and the presence of repeats.

3.1 Preparing Example Data/ Software

Begin by downloading a dedicated Github repository, which contains the reference sequences to be used in the tutorial (see https://github.com/chrishah/aMITO_MiMB; permanently archived at DOI:10.5281/zenodo.1407986), and the Illumina read data for the Oymyakon mammoth. These data are available from European Nucleotide Archive (<http://www.ebi.ac.uk/ena>) under the accession ERR852028. The download time will be around 90 min, depending on your download speed.

```
# download the dedicated repository, decompress
# and change to a new working directory
$ wget https://github.com/chrishah/aMITO_MiMB/archive/v1.0.zip
$ unzip v1.0.zip
$ cd aMITO_MiMB-1.0/tutorial/

# create the directory that will contain the read data
$ mkdir raw_reads/
# Download the Illumina read data from ENA
$ acc=ERR852028
$ wget -O raw_reads/$acc.fastq.gz \
ftp://ftp.sra.ebi.ac.uk/vol1/fastq/ERR852/$acc/$acc.fastq.gz
```

MITObim makes use of the MIRA assembler [17], as well as additional programs from the MIRA assembly suite. MITObim also ships with precompiled binaries for MIRA, if not already installed on your system. Additionally, for evaluation of the mitochondrial genome reconstructions, I recommend the BLAST+ command line applications [20]. In many Linux distributions (e.g., Ubuntu), BLAST+ is also readily available from public repositories. A range of solutions exists for multiple sequence alignment. In the below tutorial, we will be using MUSCLE [21]. On Ubuntu, MUSCLE can also be obtained from the public repository. An example for a visual tool for the exploration of NGS data is Tablet [22]. In the remainder of this section, I will detail the minimum software installation steps, assuming a recent Ubuntu distribution. Alternatively, if you decide to use the Docker container mentioned above, the following steps can be skipped, since it ships as a self-contained environment with all software (except the graphical tool Tablet) preinstalled. Instructions how to launch the Docker container are given in the dedicated Github repository (https://github.com/chrishah/aMITO_MiMB).

```
# download the MITObim repository
$ wget https://github.com/chrishah/MITObim/archive/v1.9.1.zip
$ unzip v1.9.1.zip
```

```
# test MITObim and display usage information
$ ./MITObim-1.9.1/MITObim.pl -h

# if MIRA is not installed globally you can use binaries
# that come with the MITObim repository
$ ./MITObim-1.9.1/docker/external_software/mira_4.0.2/mira -v
```

The executables for MITObim and MIRA are now stored locally in a particular location of your machine. In order to allow your commands to call these globally, i.e., from anywhere on your system without having to specify these locations, you can add them to the environmental variable “\$PATH” for the current session.

```
# add MITObim to the PATH
$ cd MITObim-1.9.1
$ PATH=$PATH:$(pwd)
# add MIRA
$ PATH=$PATH:$(pwd)/docker/external_software/mira_4.0.2
# MITObim comes with scripts that we will also add
$ PATH=$PATH:$(pwd)/misc_scripts
$ export PATH

# download and install blast+ on Ubuntu (might require 'sudo')
$ apt-get install ncbi-blast+
# test blastn and display usage information
$ blastn -h

# download and install muscle on Ubuntu (might require 'sudo')
$ apt-get install muscle
# test muscle and display usage information
$ muscle -h
```

3.2 Sequence Read Data Preprocessing and Initial Assembly

Read data used in the current tutorial has already been preprocessed. In brief, sequences have been quality controlled and filtered. Additionally, overlapping paired-end reads have been merged to achieve longer reads where possible. *See* [4] for details.

Among the most important factors for successful genome assembly is the read coverage depth (e.g., 50-fold coverage, often stated as “50×”), i.e., the number of sequence reads in which a given nucleotide position in the underlying molecule is represented. The relatively small size of animal mitochondrial genomes (~15 to 16 kbp) in combination with the mitochondrial genome being present in multiple copies per cell often results in high coverage of mitochondrial reads compared to nuclear reads in genomic data sets.

Perhaps counterintuitively, excessive coverage does not necessarily yield best assembly results. Even after rigorous quality filtering, NGS read data is by no means free of error, and high coverage

worsens the signal to noise ratio, which complicates (and sometimes worsens) assembly and results in unnecessarily long run times and increased computational requirements. Most assembly software, including the MIRA assembler, is designed for coverage depths of $\sim 70\text{--}100\times$. For best performance I therefore recommend to subsample read data to these coverage levels in order to improve the quality of your results. Furthermore, as an important strategy for the notoriously difficult assessment of genome assembly quality (especially without a reference genome), I propose repeated assembly of random subsets of data to verify results by comparison, thus leveraging extensive read coverage in mitochondrial sequence data.

Begin by performing a single MITObim iteration (“-end 0”), after which we will assess the coverage to decide for a suitable subsample size (runtime ~ 30 min).

```
# declare some variables, specify file locations, etc.
# These will be reused throughout the session
# $(pwd) inserts the full path to your present working directory
$ sample=WoollyMammoth
$ ref_id=Trichechus_manatus.AM904728.full
$ readpool=$(pwd)/raw_reads/ERR852028.fastq.gz
$ reffasta=$(pwd)/mt_refs/Trichechus_manatus.AM904728.fasta

# make and enter a new analysis directory
$ mkdir $sample.$ref_id
$ cd $sample.$ref_id

# run MITObim for a single iteration
# Note that all output will be written to the log file
$ MITObim.pl -end 0 -sample $sample -ref $ref_id \
--quick $reffasta -readpool $readpool \
--trimreads &> MITObim.log.txt
```

MITObim will generate a new directory for each iteration. In the case of the above command, a single directory “iteration0/” will have been produced that contains all intermediate files. For details on all the files produced by MIRA, please refer to the MIRA documentation. We have saved a log of the previous run to the text file “MITObim.log.txt”, which can be opened in any text editor. *M. primigenius* sequence reads will only have been baited and assembled to regions of the *T. manatus* mitochondrial genome that are sufficiently conserved (per default MIRA allows for 15% mismatch). MITObim will per default maintain the relative position between conserved regions and indicate as of yet unresolved regions in a given iteration by stretches of Ns. Output files will be given a prefix that is composed of the sample and reference name that you specified when running MITObim. For the tutorial we will

have the assembly result in fasta format displayed in the Unix shell. The assembly can also be explored at the read level in a suitable viewer, such as Tablet [22]. The assembly result can be converted into a rich data format (ace) that is natively supported by Tablet using the program `miraconvert` of the MIRA assembly suite. The resulting file “`result_iteration0.ace`” can be loaded directly into Tablet, which also summarizes basic assembly statistics. A textual summary of the assembly stats was also generated by MIRA and can be displayed in the shell.

```
# set prefix
$ prefix=$sample-$ref_id
# show assembly result in fasta format
$ cat iteration0/$prefix-it0_noIUPAC.fasta
# convert assembly to ace format for visualization in Tablet
$ it=iteration0
$ result_dir=$it/$prefix\_assembly/$prefix\_d_results
$ miraconvert $result_dir/$prefix\_out.maf \
result_iteration0.ace
# display textual summary of assembly result
$ info_dir=$it/$prefix\_assembly/$prefix\_d_info
$ cat $info_dir/$prefix\_info_contigstats.txt
```

Average read coverage is calculated across the entire sequence, so it is not very informative in this case. Maximum coverage in a few short regions is $\sim 1700\times$, but be aware that the presence of repetitive elements may locally inflate coverage. In the absence of a better estimate for overall average mitochondrial coverage, we therefore choose a cautious subsampling strategy and randomly sample 20% of the data before we proceed. MITObim provides a script that allows to draw random subsamples of the desired size in percent of the total number of reads. For every read it decides to keep/drop it based on a pseudo-random number. Specifying the seed for the pseudo-random number generator allows to exactly replicate a subsample if required.

```
# specify the location and file prefix for the new file
# containing the subsampled reads
$ outprefix=./raw_reads/ERR852028.sub20.rep01
# randomly sample ~20% of reads. Note the seed
$ downsamples.py -r $readpool -s 20 \
--seed 47271479 | gzip > $outprefix.fastq.gz
```

3.3 Assembling the Mitochondrial Genome of the Woolly Mammoth

In the following we will reconstruct the mitochondrial genome of *M. primigenius* using mt sequence data of *T. manatus* as bait for the initial assembly. I will demonstrate three different strategies that may be chosen based on the reference resources available or used

complementarily as in the current tutorial. Note that each of the following exercises assumes that you start at your base working directory.

3.3.1 Full Mitochondrial

Reference: Relative

Position Conserved

This strategy will use the full mitochondrial genome of *T. manatus* as initial bait. After the first iteration, the relative position of conserved regions will be retained, and gaps between them will be closed iteratively. This strategy is recommended if full mitochondrial reference genome resources are available and no large structural variation (large indels, inversions) are expected between the target species and the reference sequence.

We specify to perform a maximum of 500 iterations, but MITObim is expected to converge onto a stable read number well before that. Additionally we specify two optional parameters: “--trimreads” will switch on the MIRA internal read trimming routine. Using the option “--clean” will force MITObim to remove results from all but the last two iterations at any given time, which will reduce disk space. Runtime for this analysis is ~6 h.

```
# update variables and input files where necessary
# the variable $sample will be used throughout
$ sample=WoollyMammoth-sub20-rep01
$ ref_id=Trichechus_manatus.AM904728.mtgenome
$ readpool=$(pwd)/raw_reads/ERR852028.sub20.rep01.fastq.gz

# make and enter a new analysis directory
$ mkdir $sample.$ref_id
$ cd $sample.$ref_id

$ MITObim.pl -end 500 -sample $sample -ref $ref_id \
--quick $reffasta -readpool $readpool --clean \
--trimreads &> MITObim.log.txt
```

MITObim should reach a stationary read number after 47 iterations and has produced a single contiguous sequence (contig) of ~17,800 bp length. The average coverage of the final result is ~100×. You can explore the result and assembly stats as before (*see* Subheading 3.2 above). Strategies for quality assessment are demonstrated in Subheading 3.4 below.

3.3.2 Full Mitochondrial

Reference: Split Conserved
Regions

This strategy will again use the full mitochondrial genome of a related species as starting reference but does not enforce the relative position of the initially identified conserved regions. It splits (“--split”) the result into separate assembly seeds wherever resolved regions are separated by gaps. Each seed is then extended separately. To avoid spurious assemblies, we will filter out results shorter than 200bp (“--min_len 200”) and with average coverage below 50× (“--min_cov 50”). This strategy reduces the constraints

imposed on the assembly by the initial structure of the reference with respect to orientation and position of conserved regions and can be used when full mt genome reference is available, but rearrangements are expected. Runtime for this analysis is ~6 h.

```
# update variables and input files where necessary
$ ref_id=Trichechus_manatus.AM904728.split

# make and enter a new analysis directory
$ mkdir $sample.$ref_id
$ cd $sample.$ref_id

$ MITObim.pl -end 500 -sample $sample -ref $ref_id \
--quick $reffasta -readpool $readpool --clean --split \
--trimreads --min_len 200 --min_cov 50 &> MITObim.log.txt
```

MITObim reaches a stationary read number after 48 iterations and has produced five contigs. Note that the above strategy will not merge overlapping contigs automatically. We will therefore resume the process for one further iteration (starting from the assembly result of iteration 48 in maf format) and assemble the putative mitochondrial read pool identified in the previous iterations completely de novo, i.e., removing all assembly constraints imposed by the result of the previous iteration. Runtime is ~20 s.

```
# update variables and input files where necessary
$ prefix=$sample-$ref_id
$ result_dir=iteration48/$prefix\_assembly/$prefix\_d_results
$ maf=$result_dir/$prefix\_out.maf
$ readpool=iteration48/$sample-readpool-it48.fastq

$ MITObim.pl -start 49 -end 49 -sample $sample -ref $ref_id \
-maf $maf -readpool $readpool --denovo --trimreads \
--min_len 500 --min_cov 50 &> MITObim.denovo.log.txt
```

This final assembly step has produced a single contig of length ~16,900 bp. General strategies for assembly quality assessment are demonstrated in Subheading 3.4. Note that the de novo assembly step applied in the last iteration can in principle also be applied as an overall strategy for the MITObim procedure: i.e., the read pool is assembled de novo in each iteration.

3.3.3 Short Barcode Reference

This strategy uses the nucleotide sequence of a single short reference sequence as bait, which will be extended iteratively until a stable read number (and presumably a circular molecule) is reached. This will be the method of choice if only limited reference resources are available. We will use the nucleotide sequence of the

cytochrome oxidase I gene (COI) as an example bait. Runtime is ~7 min.

```
# update variables and input files where necessary
$ ref_id=Trichechus_manatus.AM904728.COI
$ reffasta=$(pwd)/mt_refs/$ref_id.nt.fasta

# make and enter a new analysis directory
$ mkdir $sample.$ref_id
$ cd $sample.$ref_id

$ MITObim.pl -end 500 -sample $sample -ref $ref_id \
--quick $reffasta -readpool $readpool --clean \
--trimreads --split --min_len 200 --min_cov 50
```

MITObim aborts after iteration 1, indicating that no reads with reasonable match to the provided reference could be found. Per default, MITObim baits reads that share at least one single sub-sequence of length $k = 31$ with the reference. COI is a relatively fast evolving gene, and it is therefore not unexpected that *M. primigenius* and *T. manatus*, which shared a MRCA some 75 million years ago, should have diverged far enough that this criterion is not fulfilled for this gene any more. We will therefore try again, but reduce the length of the minimum match to 21 bp (“--kbait 21”). Runtime is ~7 min.

```
# remove results from previous iterations
$ rm -rf iteration*

$ MITObim.pl -end 1 -sample $sample -ref $ref_id \
--quick $reffasta -readpool $readpool --clean \
--kbait 21 --trimreads --split --min_len 200 --min_cov 50
```

After one iteration, MITObim has recovered one contiguous sequence of length 548 bp. The program `blastn` of the BLAST+ command line suite of programs can be used to quickly verify that the recovered contig is a genuine fragment of COI by comparing it to the *T. manatus* COI sequence that we used as reference. Note that per default `blastn` expects at least a single perfect match of length 28 bp between query and subject. This behavior can be adjusted via the option “--word_size”, which we will set to 21 bp, according to the length k we have used before.

```
# define location of the query file
$ prefix=$sample-$ref_id
$ query=iteration1/$prefix-it1_noIUPAC.fasta
```

```
# compare contig to COI reference sequence
$ blastn -query $query -subject $reffasta -word_size 21
```

The output that is presented to us shows a full-length match with the *T. manatus* COI sequence, overall similarity of ~80%. Now that we have identified a likely genuine COI seed sequence, we can resume MITObim with the default $k = 31$. Runtime is ~9 h.

```
# specify location of maf file
$ result_dir=iteration1/$prefix\_assembly/$prefix\_d_results
$ maf=$result_dir/$prefix\_out.maf
$ MITObim.pl -start 2 -end 500 -sample $sample -ref $ref_id \
-maf $maf -readpool $readpool --clean \
--trimreads &> MITObim.log.txt
```

Using this strategy, MITObim yields a single contig of length 16,870 bp after 75 iterations.

3.4 Assessing Mitochondrial Genome Assembly Quality

Assessing the quality of genome assemblies without a reference sequence is notoriously difficult. In this section we will discuss a number of strategies particularly useful for assessing mitochondrial genome reconstruction and will demonstrate them on the result obtained in Subheading 3.3.3 above.

3.4.1 Circularity

Mitochondrial genomes are circular molecules, and the detection of circularity in the assembly result may be employed as quality criterion. Convergence to a stable number of baited reads during the iterative assembly procedure employed by MITObim may be considered a first indication of the expected circularity. In addition, MITObim provides a script that attempts to detect circularity in nucleotide sequences. For a given nucleotide sequence, the script determines all occurring unique subsequences of length k , or kmers, and their position in the original nucleotide sequence. It then reports the number of kmers that are found duplicated at each observed distance between duplicates. The occurrence of multiple kmer duplicates at the same distance that coincides with the expected length of the mitochondrial genome can be interpreted as a sign of circularity. It should be noted that the presence of tandem repeats longer than the kmer length can be problematic for accurately establishing circularity and the correct length of the circular molecule.

```
# specify variables
$ ref_id=Trichechus_manatus.AM904728.COI
$ result_dir=$sample.$ref_id/iteration75
$ fasta=$result_dir/$sample-$ref_id-it75_noIUPAC.fasta
```

```
# search for indication of circularity with k=31
$ circles.py -f $fasta -k 31
```

The script suggests clipping the sequence to a length of 16,715 bp (supported by 125 duplicated 31mers), which is in the expected length range for a mammalian mitochondrial genome. It also reports a hexa-nucleotide motif (“ACGCAT”) to be tandemly repeated ($21\times$) in the assembled sequence. Incidentally, the mitochondrial control region (D-loop) is known to contain short hexa-nucleotide tandem repeats in many vertebrates, including mammoths, where the tandem repeated motif was reported as “CGCATA” [23].

```
# clip sequence at suggested clipping points
# specify a prefix for the output via -p
$ circles.py -f $fasta -k 31 -c 0,16715 -p COI
```

The previous command creates two fasta files: one contains the putative circular nucleotide sequence clipped according to the clipping coordinates specified and a second one containing a 1000 bp fragment constructed by joining the “ends” of the putative circular sequence at its center. The latter file can be used to verify circularity, by using it as a seed for a single round of MITObim reconstruction using the putative mitochondrial read pool previously identified. If the test sequence does not represent a genuine contiguous mitochondrial sequence, we would expect the assembly result to reveal discrepancies. Note that after the initial baiting procedure, the reads will be assembled de novo, i.e., results will be based exclusively on overlap information present in sequence reads and unconstrained by the test sequence. Runtime is ~6 s.

```
# define variables and input files where necessary
$ result_dir=$(pwd)/$sample.$ref_id/iteration75
$ readpool=$result_dir/$sample-readpool-it75.fastq
$ ref_id=circularity.test.COI
$ reffasta=$(pwd)/COI.16715.for-testing.fasta
$ mkdir $sample.$ref_id
$ cd $sample.$ref_id

$ MITObim.pl -end 0 -sample $sample -ref $ref_id \
--readpool $readpool --quick $reffasta --denovo \
--trimreads &> MITObim.log.txt
```

Now, we’ll use `blastn` to compare the result of this last assembly to the seed sequence that was constructed by joining the ends of the putative circular sequence.

```
$ query=iteration0/$sample-$ref_id-it0_noIUPAC.fasta

$ blastn -query $query -subject $reffasta
```

The result is 100% identical to the test sequence, thus confirming the circularity of the nucleotide sequence assembled by MITObim.

3.4.2 Comparison to Reference Nucleotide Sequence

The most intuitive quality control for an assembly result is to compare it to a reference genome. However, MITObim allows mitochondrial genome reconstruction to be seeded by very distantly related reference sequences that may not allow for a comprehensive comparative assessment of the assembly result at the nucleotide level. Nevertheless, in the following we again use the `blastn` program to compare the assembly result to the mitochondrial genome of *T. manatus*, keeping in mind that they have diverged some 75 million years ago.

```
# specify input files
$ query=COI.circular.16715.fasta
$ reffasta=mt_refs/Trichechus_manatus.AM904728.fasta

# compare using blastn
$ blastn -query $query -subject $reffasta -word_size 21
```

The search returns three matches with 74–79% sequence identity across a cumulative length of ~16 kbp, despite the extensive evolutionary distance and indicates a high-quality genome reconstruction.

3.4.3 Gene Complement

Conservation at the protein level facilitates comparisons across greater evolutionary distances. Strictly speaking, the exact gene complement and order cannot be known a priori when assembling a previously uncharacterized mitochondrial genome. However, with some exceptions, animal mitochondrial genomes are conserved with respect to these characteristics (reviewed in [24]), and the confirmation of a standard mitochondrial gene complement can be considered a further quality criterion for mitochondrial genome assemblies.

Exact prediction of mitochondrial genes is not trivial, but a quick overview can be achieved in seconds by comparing the gene complement of a reference species to a newly assembled mitochondrial genome using the program `tblastn` of the BLAST+ command line suite, a program that is designed to compare protein against nucleotide sequences.

```
# specify input files
$ refproteins=mt_refs/Trichechus_manatus.AM904728.CDS.aa.fasta
$ ntfasta=COI.circular.16715.fasta
```

```
# compare using tblastn
$ tblastn -query $refproteins -subject $ntfasta
```

The search returns highly significant matches for 12 of the 13 mitochondrial genes (all except ATP8 match across >85% of their length) of *T. manatus*. More elaborate prediction of mitochondrial genes can be achieved, e.g., using MITOS [25], which is maintained via a webserver (<http://mitos.bioinf.uni-leipzig.de>, last accessed 16 Jan 2019). MITOS detects the full set of mitochondrial coding, rRNA and tRNA genes in our assembly result, indicating high quality.

3.4.4 Verification by Repeated Assembly with Variation

As discussed briefly in Subheading 3.2 above, an experimental approach to genome assembly may provide a powerful evaluation strategy. I therefore suggest testing assembly results for robustness across multiple technical replicates using random subsets of reads and/or a range of different parameter settings. While this approach may be impractical for large, complex nuclear genomes due to limited computational resources, such experiments are feasible on modern desktop computers for mitochondrial genomes.

In the following, we will perform multiple sequence alignment of the assembly results that we have obtained via the three strategies outlined in Subheading 3.3 using the program MUSCLE [21] and compare the results. We will first “roll” the putatively circular results obtained via the three strategies to the same starting position. We choose position 2520 in the result from Subheading 3.3.3, which was identified as starting base of tRNA Phe by MITOS, but the choice is arbitrary. Note that results from strategies in Subheadings 3.3.1 and 3.3.2 first need to be clipped as demonstrated in Subheading 3.4.1.

```
# roll result from Subheading 3.3.3 to new starting position
$ circles.py -n 2520 -p COI -f COI.circular.16715.fasta

# clip result from Subheading 3.3.1
$ ref_id=Trichechus_manatus.AM904728.mtgenome
$ result_dir=$sample.$ref_id/iteration47
$ fasta=$result_dir/$sample-$ref_id-it47_noIUPAC.fasta
$ circles.py -f $fasta -k 31 -c 0,16886 -p mtgenome
# identify the homologous starting position
$ blastn -query COI.rolled.2520.fasta \
  -subject mtgenome.circular.16886.fasta
# roll the result from Subheading 3.3.1 to position 16775
# according to the blastn result
$ circles.py -n 16775 -p mtgenome \
  -f mtgenome.circular.16886.fasta
```



```
# clip result from Subheading 3.3.2
$ ref_id=Trichechus_manatus.AM904728.split
$ result_dir=$sample.$ref_id/iteration49
$ fasta=$result_dir/$sample-$ref_id-it49_noIUPAC.fasta
$ circles.py -f $fasta -k 31 -c 0,16727 -p split
# identify the homologous starting position
$ blastn -query COI.rolled.2520.fasta \
-subject split.circular.16727.fasta
# roll the result from Subheading 3.3.2 to position 49
# according to the blastn result
$ circles.py -n 49 -p split -f split.circular.16727.fasta

# concatenate all results into a single file
$ cat *rolled* > M_primigenius.results.fasta
# align with muscle - output in fasta format
$ muscle -in M_primigenius.results.fasta \
-out M_primigenius.results.aln.fasta
```

Inspecting the multiple sequence alignment using a software solution of your choice, e.g., Jalview [26], Aliview [27], or MEGA [28], reveals minimal disagreement between assembly results. Differences are largely restricted to what appears to be the mitochondrial control region (D-loop), a section characterized by tandem repeats in many animals which is often impossible to resolve unambiguously using short-read sequencing technologies. Generally, discrepancies between technical replicates may indicate problematic regions of the assembly that can be investigated visually at the read level using appropriate visualization tools (*see* Subheading 3.4.5 below) and resolved using assembly parameter tuning that is beyond the scope of this tutorial or targeted Sanger sequencing.

3.4.5 Visual Evaluation of Assembly Results

While for large complex genomes, manual inspection of assemblies at the read level often has to be restricted to only key regions of interest, the relatively small size of animal mitochondrial genomes makes a critical visual inspection of the entire result feasible. A number of graphical visualization tools exist that allow to visually identify problematic assembly regions characterized through increased conflict at the read level. Tablet [22] is convenient for visualizing assemblies generated with MIRA because it natively supports the ace format (*see* Subheading 3.3.1 above for how to generate an assembly in ace format). IGV [29] is another widely used tool for visualization of genome assemblies but requires bam input format that is not directly supported by MIRA.

4 Note

1. The assembly of (mitochondrial) genomes is a complex and fast-developing field, and technologies, algorithms, and approaches are constantly being improved. Successful accurate reconstruction of mitochondrial genomes depends on many factors, including particular characteristics of the target genome, such as repeat content, GC content, etc. A number of tools have been developed for the assembly of organellar genomes (*see* Table 2). To date, MITObim appears to be the most widely used and has provided robust results across a wide range of taxa, but there is no “one-size-fits-all” solution to the complex problem of mitochondrial genome assembly.

Table 2
Non-exhaustive summary of existing tools specifically designed for the assembly of organellar genomes from short-read data (all last accessed 16 Jan 2019)

Short name	URL	Reference
ARC	https://ibest.github.io/ARC/	[30]
MIA	https://github.com/mpieva/mapping-iterative-assembler	[31]
MITObim	https://github.com/chrishah/MITObim	[16]
ORG.Asm	https://git.metabarcoding.org/org-asm/org-asm	–
GRAbB	https://github.com/b-brankovics/grabb	[32]
NOVOPlasty	https://github.com/ndierckx/NOVOPlasty	[33]

References

1. Green RE et al (2010) A draft sequence of the Neandertal genome. *Science* 328:710–722
2. Meyer M et al (2012) A high-coverage genome sequence from an archaic Denisovan individual. *Science* 338:222–226
3. Schubert M et al (2014) Prehistoric genomes reveal the genetic foundation and cost of horse domestication. *Proc Natl Acad Sci U S A* 111: E5661–E5669
4. Palkopoulou E et al (2015) Complete genomes reveal signatures of demographic and genetic declines in the woolly mammoth. *Curr Biol* 25:1395–1400
5. Węcek K et al (2016) Complex admixture preceded and followed the extinction of wisent in the wild. *Mol Biol Evol.* <https://doi.org/10.1093/molbev/msw254>
6. Barnett R et al (2016) Mitogenomics of the extinct cave lion, *Panthera spelaea* (Goldfuss, 1810), resolve its position within the *Panthera* cats. *Open Quaternary* 2:4
7. Lindqvist C et al (2010) Complete mitochondrial genome of a Pleistocene jawbone unveils the origin of polar bear. *Proc Natl Acad Sci U S A* 107:5053–5057
8. Llamas B et al (2016) Ancient mitochondrial DNA provides high-resolution time scale of the peopling of the Americas. *Sci Adv* 2:e1501385
9. Hervella M et al (2016) The mitogenome of a 35,000-year-old *Homo sapiens* from Europe supports a Palaeolithic back-migration to Africa. *Sci Rep* 6:25501
10. Soubrier J et al (2016) Early cave art and ancient DNA record the origin of European bison. *Nat Commun* 7:13158
11. Paijmans JLA, Gilbert MTP, Hofreiter M (2013) Mitogenomic analyses from ancient DNA. *Mol Phylogenet Evol* 69:404–416

12. Soares AER et al (2016) Complete mitochondrial genomes of living and extinct pigeons revise the timing of the columbiform radiation. *BMC Evol Biol* 16:230
13. Gansauge M-T, Meyer M (2014) Selective enrichment of damaged DNA molecules for ancient genome sequencing. *Genome Res* 24:1543–1549
14. Carpenter ML et al (2013) Pulling out the 1%: whole-genome capture for the targeted enrichment of ancient DNA sequencing libraries. *Am J Hum Genet* 93:852–864
15. Nagarajan N, Pop M (2013) Sequence assembly demystified. *Nat Rev Genet* 14:157–167
16. Hahn C, Bachmann L, Chevreux B (2013) Reconstructing mitochondrial genomes directly from genomic next-generation sequencing reads—a baiting and iterative mapping approach. *Nucleic Acids Res* 41(13): e129
17. Chevreux B, Wetter T, Suhai S et al (1999) Genome sequence assembly using trace signals and additional sequence information. *German Conf Bioinformatics* 99:45–56
18. Cock PJA, Fields CJ, Goto N, Heuer ML, Rice PM (2010) The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Res* 38:1767–1771
19. Arnason U et al (2008) Mitogenomic relationships of placental mammals and molecular estimates of their divergences. *Gene* 421:37–51
20. Camacho C et al (2009) BLAST+: architecture and applications. *BMC Bioinformatics* 10:421
21. Edgar RC (2004) MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Res* 32:1792–1797
22. Milne I et al (2013) Using tablet for visual exploration of second-generation sequencing data. *Brief Bioinform* 14:193–202
23. Rogaev EI et al (2006) Complete mitochondrial genome and phylogeny of Pleistocene mammoth *Mammuthus primigenius*. *PLoS Biol* 4:e73
24. Lavrov DV, Pett W (2016) Animal mitochondrial DNA as we do not know it: mt-genome organization and evolution in nonbilaterian lineages. *Genome Biol Evol* 8:2896–2913
25. Bernt M et al (2013) MITOS: improved de novo metazoan mitochondrial genome annotation. *Mol Phylogenet Evol* 69:313–319
26. Waterhouse AM, Procter JB, Martin DMA, Clamp M, Barton GJ (2009) Jalview Version 2—a multiple sequence alignment editor and analysis workbench. *Bioinformatics* 25:1189–1191
27. Larsson A (2014) AliView: a fast and lightweight alignment viewer and editor for large datasets. *Bioinformatics* 30:3276–3278
28. Tamura K, Stecher G, Peterson D, Filipski A, Kumar S (2013) MEGA6: molecular evolutionary genetics analysis version 6.0. *Mol Biol Evol* 30:2725–2729
29. Robinson JT et al (2011) Integrative genomics viewer. *Nat Biotechnol* 29:24–26
30. Hunter SS et al (2015) Assembly by reduced complexity (ARC): a hybrid approach for targeted assembly of homologous sequences. *bioRxiv* 014662. doi:10.1101/014662
31. Green RE et al (2008) A complete Neandertal mitochondrial genome sequence determined by high-throughput sequencing. *Cell* 134:416–426
32. Brankovics B et al (2016) GRAB: selective assembly of genomic regions, a new niche for genomic research. *PLoS Comput Biol* 12: e1004753
33. Diercksens N, Mardulyn P, Smits G (2017) NOVOPlasty: de novo assembly of organelle genomes from whole genome data. *Nucleic Acids Res* 45:e18